HACKATHON TESTING REPORT

Pothole Detection and Reporting System | Local application verification

Report date	24 October 2026, 12:29 India Standard Time	**Environment**	Windows local development
Frontend	http://localhost:5173	**Backend**	http://localhost:8000

Overall assessment: core local reporting workflows were exercised successfully. The current pothole detector did not perform adequately on the sampled annotated dataset and is not ready for reliable real-world reporting.

Executive summary

The React/Vite frontend and FastAPI backend were run locally. Backend smoke/integration checks and browser/API checks exercised reporting, lifecycle updates, media, PDF output, authority routing, and video processing. This report separates workflow success from model quality: successful API/UI operation does not mean detections are accurate.

Test results

Local services	Frontend on port 5173 and backend on port 8000 responded.	**PASS**
Backend API smoke checks	Health/config status, authorities, pothole list/statistics, and GPS-to-authority lookup returned expected responses.	**PASS**
Image reporting lifecycle	Image upload/detection, report creation, status update/history, media retrieval, resolution-proof upload, PDF report, and cleanup were exercised.	**PASS**
Video API and UI	A short clip was accepted; API analyzed 5 frames and returned 1 HIGH-severity frame detection. UI showed the frame count and an annotated keyframe that loaded at 640 x 480.	**PASS***
Video preview	The tested clip did not play in Chromium; the browser reported that it contained no supported streams. Detection response and keyframe display still worked.	**LIMITED**
Kaggle sample detection	12 annotated images: 31 ground-truth boxes; model returned 21 boxes. At IoU >= 0.3, 1 prediction matched: approximate precision 4.8%, recall 3.2%. All 12 images were assigned CRITICAL as highest severity.	**FAIL QUALITY**
Email / cloud storage	Email delivery is simulated without SMTP credentials; S3 is not configured. These services were not verified live.	**NOT CONFIGURED**

*Pass refers to API result and rendered annotated keyframe, not playback of this specific clip.

Detection quality and interpretation

The Kaggle evaluation used a 12-image sample from the Andrew MVD pothole dataset (VOC bounding-box annotations; 665 images in the source dataset). The figures are sample-level results, not a statistically comprehensive benchmark. With only one match at IoU 0.3, the present model should be treated as a prototype; detections and severity labels require human review. The bundled yolov8n.pt is a general-purpose COCO model, not a pothole-trained model.

Credentials and API-key guidance

No general AI API key is required for the local app. Detection uses the local YOLO model. Maps use Leaflet/OpenStreetMap tiles and do not require a key in this setup. VITE_API_URL is an API base URL setting, not a secret; it can be left blank for local development because Vite proxies requests to localhost:8000.

Optional integrations (only configure these if needed):

Live email notifications	A Gmail App Password (not your normal account password), or credentials from another SMTP provider.	Backend/.env: SMTP_USER, SMTP_PASSWORD, SMTP_HOST, SMTP_PORT, NOTIFICATION_SENDER_EMAIL. Never place SMTP secrets in the frontend.
S3 object storage	AWS IAM access key ID + secret, bucket, and region; grant only required bucket permissions.	Backend/.env: USE_AWS_S3=true, AWS_S3_BUCKET_NAME, AWS_REGION, AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY.
Pothole-trained model	A compatible YOLO weights file; this is a model artifact, not an API key.	Set MODEL_PATH in Backend/.env to the model file path. Do not overwrite or claim an unverified model improves accuracy.

Create Backend/.env from Backend/.env.example if it does not already exist; do not overwrite an existing file. Keep credentials out of Git and out of all VITE_* variables, because frontend variables are exposed to browser users. No credential was generated or included in this report.

Run locally

Backend (PowerShell)

```
cd "C:\Users\abhishek pandey\Downloads\SadakSignal\Pothole-Detection-and-Reporting-System\Backend"
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app --reload
```

Frontend (second PowerShell)

```
cd "C:\Users\abhishek pandey\Downloads\SadakSignal\Pothole-Detection-and-Reporting-System\Frontend"
npm run dev
```

Recommended before production: use a validated pothole-trained model and evaluate on a larger held-out set; recalibrate severity; test supported video codecs; configure and verify SMTP/storage only if required. This was a local test, not a public deployment.

Evidence note: this summary reflects the project smoke/integration checks and browser/API observations recorded during the local verification. Generated test files were cleaned from the uploads folder afterward. Kaggle dataset source credit: MakeML; source dataset license is DbCL 1.0.